

C++ Pointers & References — Placement-Oriented Notes

1. Pointer — Basic Concept

A pointer is a variable that stores the address of another variable.

```
int x = 10;
```

```
int *p = &x;
```

```
cout << x << endl;    // 10
```

```
cout << &x << endl;  // address of x
```

```
cout << p << endl;    // address stored in p
```

```
cout << *p << endl;  // 10
```

Where is it used?

- Dynamic memory allocation
- Arrays and strings
- Functions
- Data structures
- Linked lists, trees, graphs
- Passing data efficiently
- Low-level/system programming

Why is it used?

Because sometimes we don't want to work with the **value directly**. We want to work with its **memory address**.

How?

Three important operators:

Operator	Meaning
----------	---------

&x	Address of x
----	--------------

p	Address stored in pointer
---	---------------------------

*p	Value at that address
----	-----------------------

Think:

x = 10

Memory

+-----+

| 10 |

+-----+

↑

|

p

2. Types of Pointers

There isn't just one "type of pointer." In placement interviews, students should know the important categories.

Null Pointer

A null pointer intentionally points to nothing.

Modern C++:

```
int *p = nullptr;
```

Where is it used?

When a pointer currently has **no valid object/address** to point to.

Very common in:

- Linked lists
- Trees
- Searching
- Dynamic memory
- Error checking

Example:

```
int *p = nullptr;
```

```
if(p == nullptr)
```

```
{
```

```
    cout << "Pointer does not point to anything";
```

```
}
```

Why?

It is safer than leaving a pointer uninitialized.

How?

```
int *p = nullptr;
```

Avoid:

```
int *p;
```

because p contains an unpredictable address.

Placement question

Q: Why is nullptr preferred over NULL in modern C++?

Because nullptr has its own type, `std::nullptr_t`, and avoids some ambiguity with integer 0.

Wild Pointer

A wild pointer is an uninitialized pointer.

```
int *p;
```

```
cout << *p; // Dangerous
```

p contains an unknown address.

Where?

Usually nowhere intentionally. It is a programming mistake.

Why?

It happens because the programmer forgot to initialize the pointer.

How to prevent it?

```
int *p = nullptr;
```

Interview difference

```
int *p;           // Wild pointer
```

```
int *p = nullptr; // Null pointer
```

```
#include<iostream>
```

```
using namespace std;
```

//Whenever we are dealing with a pointer, if a pointer is created but not assigned to `null`, then it's a wild pointer. When assigned to `null`, `ptr` becomes a null pointer. We prefer creating a null pointer.

```

int main()
{
    int *p;//wild pointer:A pointer which is created but not referring to anything would have
garbage inside, called a wild pointer.
    cout<<"\nPoints to:"<<p;
    int *p2=nullptr;
    cout<<"\nPoints to:"<<p2;
    return 0;
}

```

Dangling Pointer

This is extremely important for interviews.

A dangling pointer points to memory that is no longer valid.

Example:

```
int *p = new int(10);
```

```
delete p;
```

```
cout << *p; // Dangerous
```

After:

```
delete p;
```

the memory has been released, but p still contains the old address.

So:

p —————→  released memory

Where does it occur?

Common situations:

- After delete
- Returning address of a local variable
- Object destruction
- Invalid references/pointers

Why is it dangerous?

Because accessing it produces **undefined behavior**.

How to prevent it?

```
delete p;
```

```
p = nullptr;
```

Now:

p → nullptr

Placement favourite

Q: What is the difference between a wild pointer and dangling pointer?

Wild

Never initialized properly

Contains garbage/unpredictable address

int *p;

Exists due to initialization problem

Dangling

Was valid earlier

Contains address of invalid memory

delete p;

Usually occurs after lifetime ends

```
#include<iostream>
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    int *p=new int(10);
```

```
    cout<<"\n data:"<<*p<<" at address:"<<p;
```

```
    delete p;
```

```
    p=nullptr;//This will make a pointer null, stopping the dangling pointer.
```

//When a pointer is deleted or the allocated dynamic memory is removed, it behaves in a wild manner called dangling. This means it can randomly refer to any location and will show any random data.

```
    cout<<"\n data:"<<*p<<" at address:"<<p;
```

```
    //This type of pointer is called a dangling pointer as it is in a dilemma.
```

```
        return 0;
```

```
}
```

Void Pointer

A void* can store the address of different types.

```
int x = 10;
```

```
void *p = &x;
```

But you cannot directly dereference it.

```
cout << *p; // Error
```

You need type conversion:

```
cout << *(int*)p;
```

Where is it used?

Historically and in low-level/generic C-style APIs.

Why?

Because it can hold the address of different data types.

How?

```
int x = 10;
```

```
float y = 5.5;
```

```
void *p;
```

```
p = &x;
```

```
cout << *(int*)p;
```

```
p = &y;
```

```
cout << *(float*)p;
```

Placement point

void* knows the **address**, but not the **data type**.

Therefore the compiler doesn't know:

- How many bytes to read
- How to interpret those bytes

```
#include<iostream>
```

```
//void:A single pointer which can refer to
```

```
//any data type but requires casting at the time of access
```

```
//This is because the pointer will point to the start of the memory address but how many
```

```
//bytes to read would be decided by the cast.
```

```
using namespace std;
```

```
int main()
```

```
{
```

```
    void *p;
```

```
    int x=10;
```

```
    float y=3.14;
```

```
    p=&x;
```

```
    cout<<"\n P points to:"<<*(int *)p;
```

```
    p=&y;
```

```
    cout<<"\n P points to:"<<*(float *)p;
```

```
    return 0;
}
```

Pointer to Constant

```
int x = 10;
```

```
const int *p = &x;
```

You can change x:

```
x = 20;
```

But you cannot change x through p:

```
*p = 30; // Error
```

Where?

When a function should be able to **read data but not modify it**.

Why?

To protect data from accidental modification.

How?

```
void display(const int *p)
```

```
{
    cout << *p;
}
```

8. Constant Pointer

Now look carefully:

```
int x = 10;
```

```
int y = 20;
```

```
int *const p = &x;
```

Here the **pointer itself cannot change**.

```
*p = 50; // Allowed
```

```
p = &y; // Error
```

Easy memory trick

```
const int *p;
```

Data is constant through pointer.

```
int *const p;
```

Pointer is constant.

And:

```
const int *const p = &x;
```

Both are constant.

```
#include<iostream>
```

```
//Constant pointer : const--creates constant
```

```
//I can create a pointer which is constantly referring to a single address.
```

```
//I can create a pointer, which is a constant, using which you cannot change data.  
using namespace std;
```

```
int main()
```

```
{
```

```
    /*const float pi=3.14;//Pi would be constant and cannot be changed once created.
```

```
    cout<<"\npi:"<<pi;
```

```
    pi=3.10;
```

```
    cout<<"\npi:"<<pi;
```

```
    */
```

```
    int x=10,y=20;
```

```
    int *const p=&x;//You created a constant pointer which can only refer to a single variable  
    via its address throughout its lifecycle.
```

```
    cout<<"\n p points to:"<<*p;
```

```
    const int *p2=&x;//P2 is a special type of pointer which can access any integer value but  
    will not be able to change it.
```

```
    cout<<"\n p2 points to:"<<*p2;
```

```
    p2=&y;
```

```
    cout<<"\n p2 points to:"<<*p2;
```

```
    *p2=*p2+5;
```

```
    return 0;
```

```
}
```

Pointer vs Reference

This is perhaps the most important conceptual comparison.

Suppose:

```
int x = 10;
```

Pointer

```
int *p = &x;
```

Reference

```
int &r = x;
```

Both can access x.

```
*p = 20;
```

```
r = 30;
```

Now:

```
cout << x;
```

Output:

30

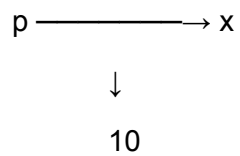
Pointer vs Reference — The Easy Explanation

Tell students:

A pointer is an address stored in a variable. A reference is another name/alias for an existing variable.

Think:

Pointer:



Reference:



But conceptually r is simply **another name for x**.

Major Differences

Feature	Pointer	Reference
Syntax	int *p	int &r
Stores address	Yes	Conceptually an alias
Can be nullptr	Yes	No
Can be uninitialized	Yes	Must be initialized
Can change what it refers to	Yes	No, once bound
Dereferencing required	Yes	No
Pointer arithmetic	Yes	No
Can represent "nothing"	Yes	No
Can have pointer to pointer	Yes	References have different rules
Commonly used for	Address/memory manipulation	Aliasing/pass-by-reference

pointer was referring to data by its address but this created security issues and hence it was closed.

reference:Reference is a modernized solution to the pointer but in real life it creates an alias.
/Alternate identifier.

It is created and assigned only once during its life cycle and will be accessing data directly at the memory level without opening the address.

```
int chandan=10;
```

```
int &chandu=chandan;//Now Chandu and Chandan both are exactly referring to one memory location.
```

Chandan changes, Chandu reflects Chandu changes, Chandan reflects.

//Reference is established and created only once throughout the program and is maintained throughout the program.

```
int *p;----->int &p=<something>;
```

Pointer to String

In C++, a string can be represented using a character array.

```
char str[] = "Hello";
```

```
char *p = str;
```

```
cout << p << endl;
```

Output:

Hello

Why?

Because p points to the first character.

str

H e l l o \0

↑

p

You can access characters:

```
cout << *p << endl;    // H
```

```
cout << *(p + 1) << endl; // e
```

```
cout << *(p + 2) << endl; // l
```

Where?

- String processing
- Character manipulation
- C-style APIs
- Interview questions involving arrays and pointers

Why?

Because a character array and pointer can be traversed using pointer arithmetic.

How?

```
char str[] = "Hello";
```

```
char *p = str;
```

```
while(*p != '\0')
```

```
{
    cout << *p;
    p++;
}
```

Output:

Hello

10. Pointer to String Literal

This is a **very important C++ interview trap**.

Consider:

```
const char *p = "Hello";
```

Here "Hello" is a **string literal**.

p points to the first character of that literal.

"Hello"

H e l l o \0

↑

p

You should treat the literal as non-modifiable.

Therefore:

```
const char *p = "Hello";
```

is appropriate.

Do **not** write:

```
char *p = "Hello";
```

as the modern C++ approach.

And definitely don't do:

```
*p = 'X';
```

because modifying a string literal results in undefined behavior.

Important distinction

```
char str[] = "Hello";
```

creates a **modifiable character array**.

```
const char *p = "Hello";
```

points to a **string literal that should not be modified**.

This distinction is frequently tested.

11. Pointer to Array

Since you already covered the basic concept, teach students the exact syntax difference.

```
int a[5] = {10,20,30,40,50};
```

```
int (*p)[5] = &a;
```

Read:

p is a pointer to an array of 5 integers.

Access:

```
cout << (*p)[0];
```

Output:

10

Where?

Especially useful with:

- 2D arrays
- Functions
- Matrix processing
- Dynamic multidimensional arrays

Why?

Because sometimes we need a pointer that represents the **entire array**, not merely its first element.

Critical interview difference

```
int *p = a;
```

means:

pointer to an integer

Whereas:

```
int (*p)[5] = &a;
```

means:

pointer to an array of 5 integers

12. Pointer to Pointer / Double Pointer

You have already covered this, but students should understand the purpose.

```
int x = 10;
```

```
int *p = &x;
```

```
int **q = &p;
```

Memory:

```
x = 10
```

```
q
```

```
↓
```

```
p
```

```
↓
```

```
x
```

```
↓
```

```
10
```

Therefore:

```
cout << x;    // 10
```

```
cout << *p;   // 10
```

```
cout << **q;  // 10
```

Where?

- Dynamic 2D structures
- Modifying a pointer inside a function
- Linked lists
- Trees
- Dynamic memory structures
- Command-line arguments

Why?

Because sometimes we need the **address of a pointer**.

13. Dynamic Memory Allocation

This is one of the most important reasons pointers exist.

Normally:

```
int x = 10;
```

Memory is automatically managed.

But sometimes we need memory **at runtime**.

C++ provides:

`new`

`delete`

`new[]`

`delete[]`

Dynamic Single Variable

```
int *p = new int;
```

```
*p = 10;
```

```
cout << *p;
```

```
delete p;
```

```
p = nullptr;
```

Where?

When memory size/lifetime needs to be decided during program execution.

Why?

Because memory requirements may not be known at compile time.

How?

```
int *p = new int(10);
```

This:

1. Allocates memory
2. Stores 10
3. Returns its address
4. Stores that address in p

14. Dynamic Array

Suppose the user decides the size.

```
int n;
```

```
cin >> n;
```

```
int *arr = new int[n];
```

```
for(int i = 0; i < n; i++)
```

```
{
```

```
    cin >> arr[i];
```

```
}
```

```
for(int i = 0; i < n; i++)
```

```
{
```

```
    cout << arr[i] << " ";
```

```
}
```

```
delete[] arr;
```

```
arr = nullptr;
```

Example

Input:

5

10 20 30 40 50

Output:

10 20 30 40 50

Placement concept

Students should understand:

new

pairs with:

delete

and

new[]

pairs with:

`delete[]`

Wrong pairing can lead to undefined behavior.

15. Why Dynamic Memory?

Suppose:

```
int arr[1000];
```

You allocate 1000 elements whether you need them or not.

But:

```
int n;
```

```
cin >> n;
```

```
int *arr = new int[n];
```

allows the size to be decided at runtime.

Conceptually:

Compile time



Program starts



User enters size



Memory allocated



Use memory



`delete[]`

16. Memory Leak

Another placement favourite.

```
int *p = new int(10);
```

Suppose you forget:

```
delete p;
```

The allocated memory is no longer properly accessible through your program, but it remains allocated.

This is a **memory leak**.

Where?

Long-running programs are particularly affected.

Why?

Repeated memory leaks can consume available memory.

How to prevent?

`delete p;`

`p = nullptr;`

For arrays:

`delete[] arr;`

`arr = nullptr;`

Modern C++ note

In real-world modern C++, prefer **RAII and smart pointers** such as:

`unique_ptr`

`shared_ptr`

rather than manually managing ownership with `new/delete` whenever possible.